

CSCE 614 COMPUTER ARCHITECTURE · MODULE 07 · LECTURE NOTES

Virtual Memory and the TLB

The indirection under every memory access.

MODULE OUTCOMES

- Explain address translation and multi-level page tables.
- Explain the TLB and why it is on the critical path.
- Quantify TLB reach and recognise when it is exceeded.
- Explain huge pages and when they help.
- Explain page faults, swapping, and memory-mapped files.

1 Why virtual memory

Programs do not use physical addresses. They use **virtual** addresses, translated by hardware into physical ones on every access. One indirection buys four distinct things:

- **Isolation.** A process cannot even name memory belonging to another, because the translation simply does not exist. Memory protection is a consequence of the addressing scheme rather than a check bolted on top.
- **Relocation.** Every program can be compiled as though it starts at the same address, and be placed anywhere physically.
- **Over-subscription.** The sum of all processes' address spaces may exceed physical memory, with inactive pages evicted to disk.
- **Sharing.** A single physical copy of a shared library or a file can be mapped into many address spaces at once.

1.1 Multi-level page tables

x86-64 virtual address (48 significant bits):

47	39	38	30	29	21	20	12	11	0		
+-----+-----+-----+-----+-----+											
PML4			PDPT			PD		PT		OFFSET	
+-----+-----+-----+-----+-----+											
9			9			9		9		12 bits	

A flat page table for a 48-bit address space with 4 KB pages would need 2^{36} entries — hundreds of gigabytes per process. A multi-level table is sparse: a region of address space that is never used costs nothing because the corresponding upper-level entry is simply absent.

The cost is that translating one address requires walking four levels, and each level's location depends on the previous level's contents. These are four **dependent** memory accesses — exactly the serialised pointer chasing of Module 02, and equally unamenable to overlapping. At 250 cycles each when uncached, a full walk approaches a thousand cycles, before the actual data access begins.

2 The TLB

A thousand cycles per access is obviously impossible, so translations are cached in the **Translation Lookaside Buffer**.

Property	Typical value
L1 data TLB entries	64–100, often fully associative
L2 (shared) TLB entries	1,500–2,000
Hit cost	Effectively zero — overlapped with cache lookup
Miss cost	A page walk: up to four dependent memory accesses, partially cached in dedicated page-walk caches

The TLB sits on the critical path of every memory access, which is why it is kept small and fast, and why its capacity becomes a real constraint.

2.1 TLB reach

$$\text{reach} = \text{TLB entries} \times \text{page size}$$

Entries	Page size	Reach
1,536	4 KB	6 MB
1,536	2 MB	3 GB
1,536	1 GB	1.5 TB

THE TLB DOES NOT REACH AS FAR AS YOUR CACHE

A modern L3 cache can be 32 MB or more. With 4 KB pages, the TLB reaches around 6 MB.

So a 20 MB working set *fits in L3* and still incurs a page walk on most accesses outside the first 6 MB. The data is in cache; the *translation* is not.

The symptom is distinctive and easily misdiagnosed: a program that is not bandwidth-bound, has a low L3 miss rate, and is nonetheless far slower than it should be. The counter to check is dTLB-load-misses.

This is why there are effectively **two** hierarchies to satisfy: the data cache hierarchy and the translation hierarchy. Code can be perfect for one and hostile to the other.

3 Huge pages

Larger pages multiply TLB reach directly and shorten the page walk by removing a level.

Page size	Benefit	Cost
2 MB	512× the reach of 4 KB pages; one fewer level in the walk.	Internal fragmentation: a mostly-empty 2 MB page wastes real memory. Harder to allocate once memory is fragmented.
1 GB	Enormous reach; appropriate for very large data structures.	Only sensible for large, long-lived, densely used regions. Usually reserved at boot.
Transparent huge pages	Automatic promotion, no code changes.	The allocator may stall trying to assemble a contiguous 2 MB region, producing unpredictable latency spikes. Several databases recommend disabling THP for this reason.

Hardware prefetchers also stop at page boundaries, since prefetching into an unmapped page risks a spurious fault. Larger pages therefore let prefetching run further uninterrupted — a secondary benefit that is easy to overlook.

4 Page faults

Kind	Cause	Typical cost
Minor	The page is in physical memory but not mapped in this process's tables — first touch of newly allocated memory, copy-on-write, or a shared page already resident.	~1 μ s. Routine.
Major	The page must be read from storage.	~100 μ s on SSD; ~10 ms on rotating disk. Visible to users.
Invalid	No valid mapping exists for this address.	SIGSEGV. This is a bug in your program, not a performance issue.

4.1 Consequences worth knowing

- **First touch is slow.** malloc reserves address space; physical pages are allocated lazily on first write. This is a large part of why benchmark warm-up matters (Module 01) — the first pass pays for every page.
- **fork() is cheap.** The child shares the parent's pages copy-on-write; physical copying happens only for pages that are actually written.
- **mmap turns file I/O into memory access.** The page cache handles reading on demand, and the same physical pages serve every process mapping the file.
- **NUMA first touch.** On multi-socket systems, a physical page is allocated in the memory attached to whichever core first touches it. Allocating and initialising a large array on one thread and then processing it on many places all of it on one socket, with every other socket paying remote-access latency. **Initialise in**

parallel, with the thread that will use the data. This is a common and expensive bug in parallel code, and CSCE 735 returns to it.

5 Where this fits

TWO HIERARCHIES, NOT ONE

Modules 05 and 06 treated memory access as a question of cache behaviour, silently assuming a physical address was available. Obtaining that address is itself a memory access problem with its own cache and its own miss penalty.

A program must satisfy both hierarchies. Cache-friendly code with a scattered page footprint will thrash the TLB; TLB-friendly code with a bad stride will thrash the cache.

Diagnostic order: confirm it is memory-bound (low IPC), check cache miss rates, and if those look acceptable, check dTLB-load-misses before looking anywhere else.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Onur Mutlu — Virtual Memory lectures	Translation, TLBs, and the interaction with caches.
Ulrich Drepper — What Every Programmer Should Know About Memory, section 4	Virtual memory and TLB behaviour with measurements.
OSTEP — Virtual Memory chapters (free book)	The operating-system side: paging policy, replacement, swapping. Overlaps usefully with CSCE 611.
Linux kernel documentation — Transparent Hugepage Support	How THP works in practice, and why some workloads disable it.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Measure TLB reach: access one byte per page across a working set that grows from 1 MB to 1 GB, and find the size at which cost per access jumps. Compare with entries $\times$ 4 KB from your CPU's specification.
- 2 Repeat the experiment with huge pages enabled (`madvise(MADV_HUGEPAGE)` or `mmap` with `MAP_HUGETLB`) and confirm the jump moves.

- 3 Construct a workload with a 20 MB working set that fits in L3 but exceeds TLB reach. Show with hardware counters that L3 misses are low while TLB misses are high.
- 4 Measure the cost of first touch: allocate 1 GB, time a first pass writing every page, then time a second pass. Explain the difference.
- 5 Measure `fork()` on a process with a 1 GB heap, then measure it again after the child writes to every page. Explain both numbers.
- 6 On a NUMA machine if you have access to one: initialise a large array from a single thread, then process it from all threads; then initialise in parallel and repeat. Report the difference.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the four things virtual memory provides.
- 2 Why are multi-level page tables used instead of a flat table, and what does the structure cost?
- 3 Why is a page walk particularly expensive beyond simply being four accesses?
- 4 Compute TLB reach for 1,536 entries at 4 KB and at 2 MB. Compare with a 32 MB L3 cache.
- 5 Describe a program that fits entirely in cache yet is limited by TLB misses. How would you detect it?
- 6 Give two costs of huge pages.
- 7 What is NUMA first touch, and what practice does it imply for parallel initialisation?